Atelier 1 de Deep Learning

Régression des prix de clôture NYSE avec un MLP PyTorch

Étudiant	Badr BERQIA
Partie rendue	Partie 1 - Régression
Dataset	NYSE - prices-split-adjusted.csv
Environnement	Windows local, Python 3.12, PyTorch, CUDA, RTX 3070

Dans ce rapport, je présente uniquement la partie régression. Je n'inclus pas la classification multiclasse de la partie 2 dans ce rendu.

Note sur mon environnement. L'énoncé mentionne Google Colab ou Kaggle comme environnements possibles. J'ai effectué tous mes entraînements et toutes mes évaluations localement sous Windows avec mon GPU personnel. J'utilise CUDA pour accélérer les calculs, tout en gardant mon notebook exécutable sur CPU.

1. Objectif et formulation du problème

Mon objectif est de prédire, pour une action donnée et un jour de cotation t, le prix de clôture ajusté au prochain jour de cotation t+1. Je considère donc chaque couple action-date comme une observation. Ma table contient 851 264 lignes, 501 symboles et sept colonnes sur la période du 4 janvier 2010 au 30 décembre 2016.

Variables utilisées

- Entrées que j'utilise : open, high, low, close et volume observés au jour t.
- Cible que je prédis : target_next_close, soit la clôture du prochain jour de cotation pour le même symbole.
- Construction que j'applique : tri par symbole et date, puis groupby("symbol")["close"].shift(-1).
- Taille finale obtenue : 850 763 exemples après retrait des 501 dernières lignes sans cible.

Contrôles de qualité

Je ne détecte aucune valeur manquante dans la table d'origine ni aucun doublon date-action. J'ai aussi vérifié que les contraintes OHLC sont respectées. Je conserve les vingt et une lignes dont le volume est nul, car elles sont rares et leurs prix restent cohérents.

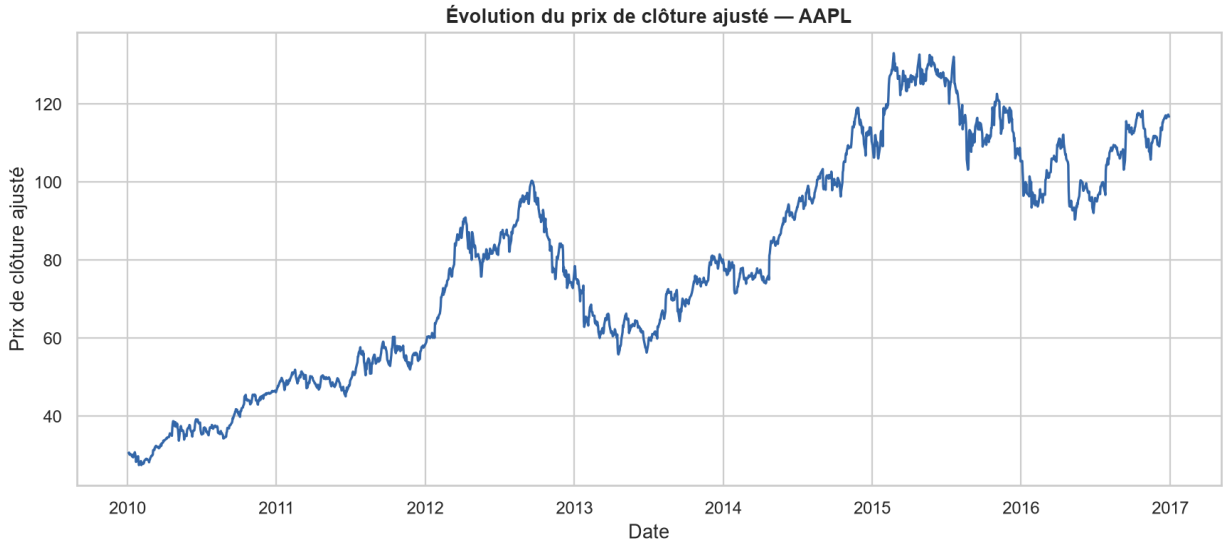


Figure 1 - Évolution temporelle du prix de clôture ajusté de l'action AAPL.

2. Analyse exploratoire

Je visualise une seule action, AAPL, afin d'éviter de superposer 501 séries temporelles. J'observe une tendance globale haussière, mais aussi plusieurs baisses locales : je ne peux donc jamais connaître le prochain cours avec certitude.

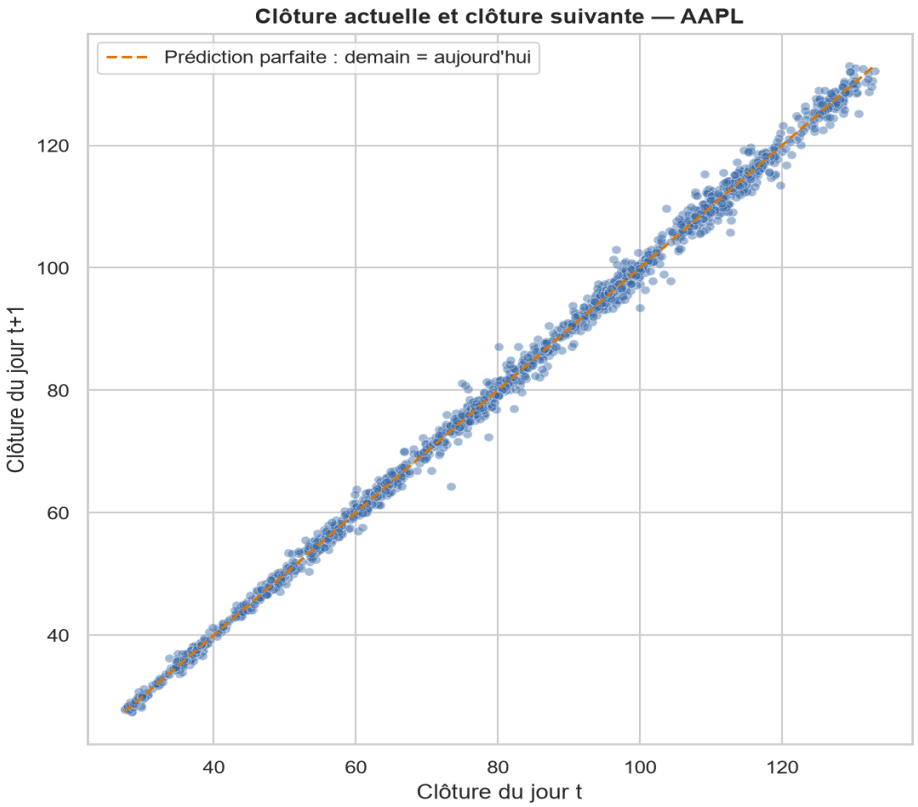


Figure 2 - La clôture du lendemain est généralement proche de la clôture courante.

J'observe que le nuage se concentre autour de la diagonale y = x. Cela justifie mon choix d'une baseline de persistance : prédire demain avec le prix d'aujourd'hui. Sur AAPL, j'obtiens une MAE de 0,9278 et une RMSE de 1,3393 avec cette règle.

3. Préparation des données

J'effectue un découpage chronologique. Je recalcule la cible séparément dans chaque période afin d'empêcher la première observation d'une période future de devenir la cible de la période précédente.

Ensemble	Période	Observations
Entraînement	2010-01-04 au 2014-12-30	599 692
Validation	2015-01-02 au 2015-12-30	124 458
Test	2016-01-04 au 2016-12-29	125 624

J'ajuste les StandardScaler des entrées et de la cible uniquement sur l'entraînement. Je transforme ensuite la validation et le test avec ces mêmes paramètres. Je convertis les arrays en float32 et je les regroupe en mini-batches PyTorch.

4. Modèle MLP initial

- J'ai choisi l'architecture 5 -> 64 -> ReLU -> 32 -> ReLU -> 1.
- Mon modèle contient 2 497 paramètres entraînables.
- J'utilise l'erreur quadratique moyenne sur la cible standardisée comme fonction de coût.
- J'utilise Adam avec un learning rate de 0,001.
- J'entraîne pendant 10 époques et je choisis le meilleur état sur la validation.

Mon réseau reçoit cinq valeurs par exemple. Ses couches cachées apprennent des représentations intermédiaires ; sa dernière couche produit une seule valeur car ma cible est un prix continu unique.

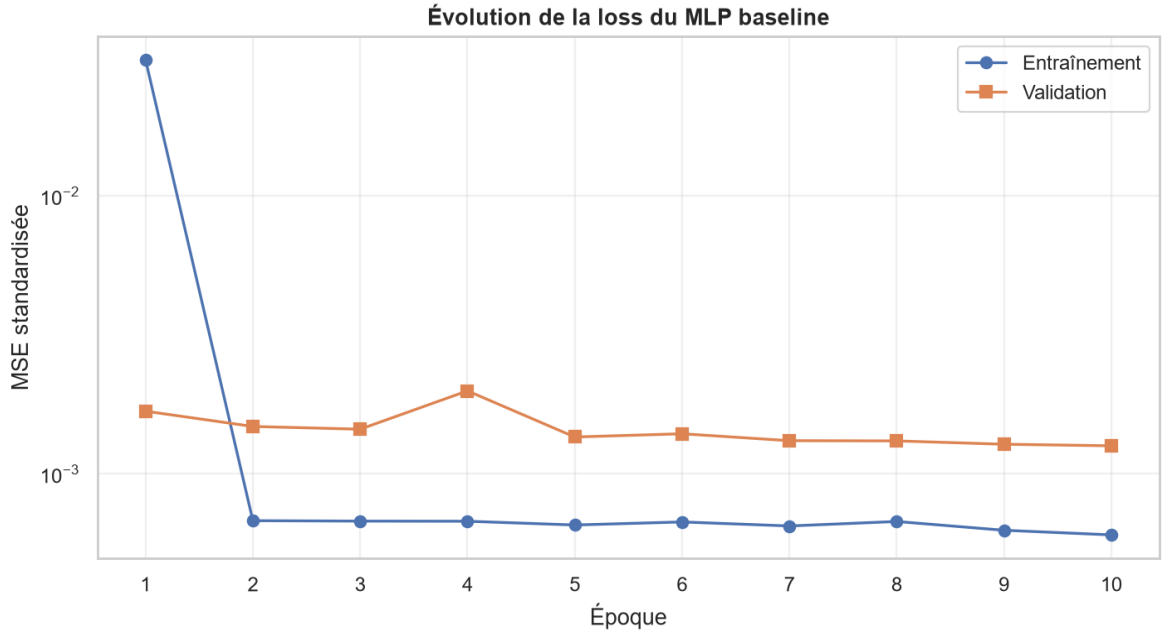


Figure 3 - Loss d'entraînement et de validation du MLP initial.

5. GridSearch et régularisation

Recherche d'hyperparamètres

Avec GridSearchCV, je compare 16 combinaisons : architectures (32,16) ou (64,32), learning rates 0,0005 ou 0,001, optimiseurs Adam ou RMSprop, et 5 ou 10 époques. Pour maîtriser le temps de calcul, j'utilise un sous-échantillon déterministe de 80 000 exemples d'entraînement et 20 000 exemples de validation.

La meilleure configuration que j'obtiens sur ce sous-échantillon est (32, 16), learning rate 0.0010, optimiseur RMSprop et 10 époques, avec une MSE de validation standardisée de 0.001160.

Après mon réentraînement complet, cette configuration ne dépasse pas mon MLP initial. J'en conclus qu'un classement sur sous-échantillon n'est pas garanti de se conserver exactement sur toutes les données.

Techniques de régularisation

- L2 : weight_decay = 10^-4 ; pénalise les poids de grande amplitude.
- Dropout : taux 0,2 après chaque couche cachée ; désactive aléatoirement des activations pendant l'entraînement.
- Dropout + L2 + early stopping : combinaison des deux pénalités et arrêt après trois époques sans amélioration.

Dans mes expériences, la régularisation n'améliore pas les performances. Le dropout dégrade fortement mon accuracy à ±5 %, ce qui indique plutôt du sous-apprentissage que la correction d'un overfitting.

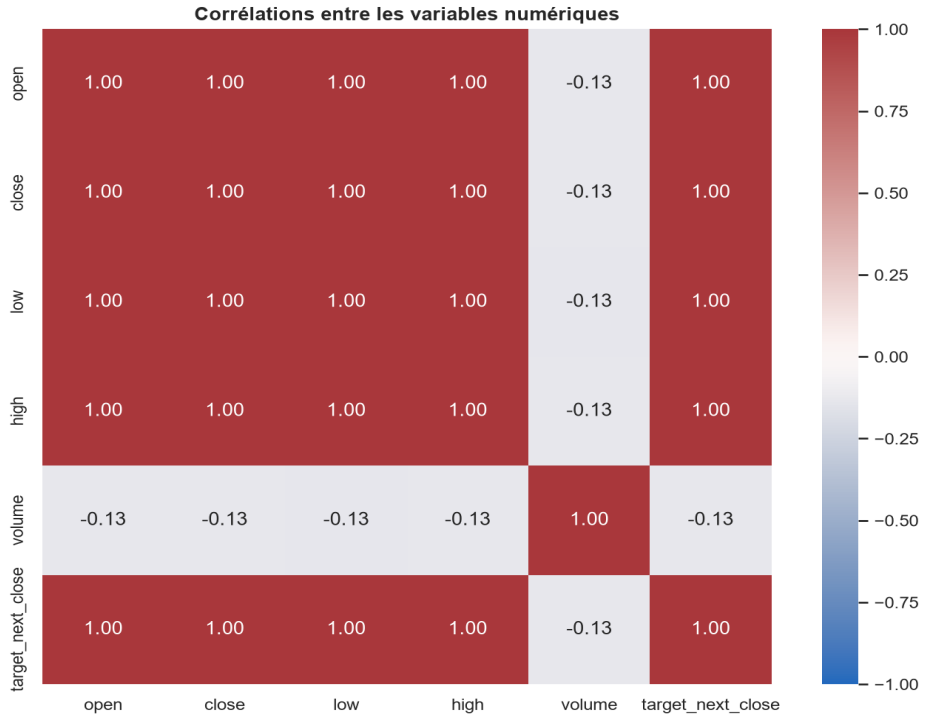


Figure 4 - Corrélations entre les variables numériques. Les prix OHLC sont très redondants.

6. Résultats sur l'ensemble de test

J'exprime la MAE et la RMSE dans l'unité du prix. Je définis l'accuracy de régression comme le pourcentage de mes prédictions dont l'erreur absolue est inférieure ou égale à 5 % de la vraie cible.

Méthode	MAE	RMSE	R^2	Acc. ±5 %
Persistance	0.9484	2.1429	0.9996	97.81 %
MLP initial	0.9811	2.1839	0.9996	97.22 %
MLP GridSearch	1.0479	2.2899	0.9995	96.60 %
L2	1.0480	2.2937	0.9995	96.92 %
Dropout	1.6114	2.8647	0.9992	83.33 %
Dropout + L2 + early stopping	1.3192	2.6681	0.9993	92.52 %

Mon résultat principal : j'obtiens la meilleure MAE (0,9484), la meilleure RMSE (2,1429) et la meilleure accuracy à ±5 % (97,81 %) avec la persistance. Mon MLP initial est proche, mais il n'apporte aucun gain qui justifierait sa complexité.

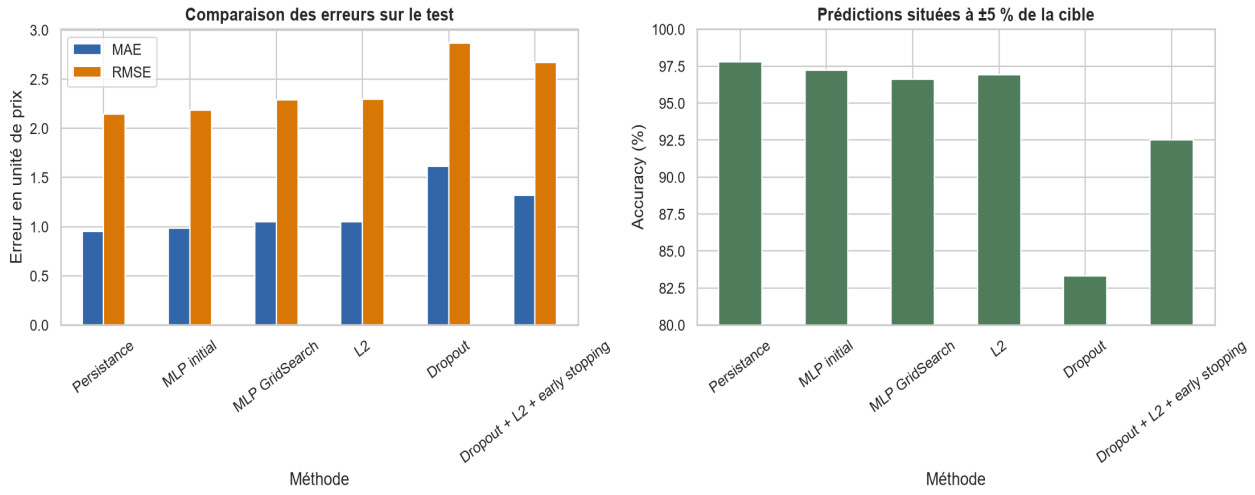


Figure 5 - Comparaison des erreurs et de l'accuracy à ±5 %.

7. Évolution au fil des époques

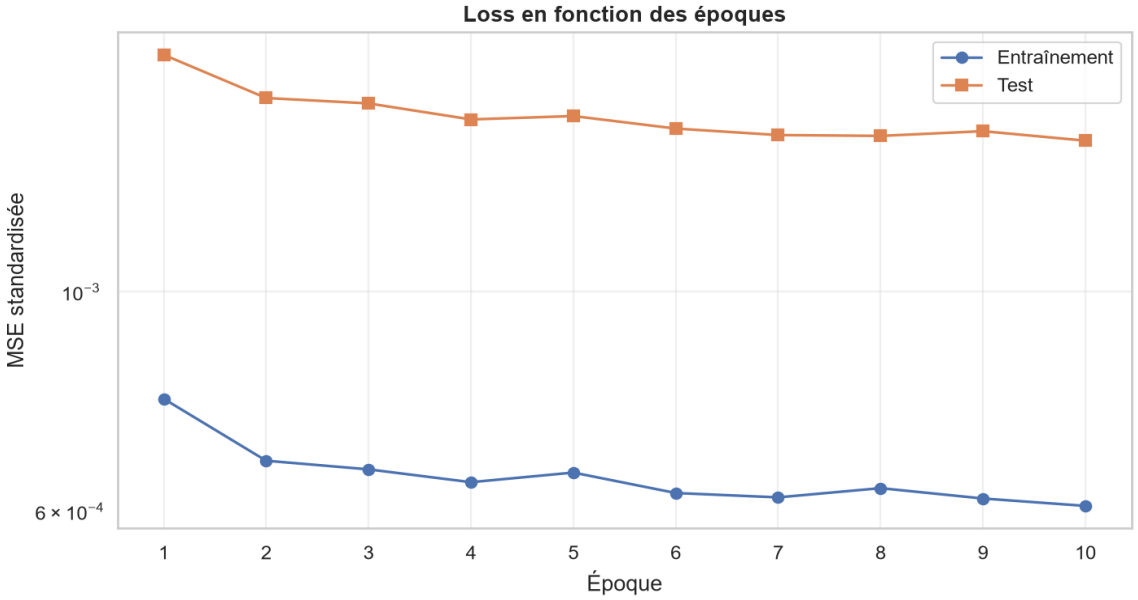


Figure 6 - Loss standardisée de l'entraînement et du test.

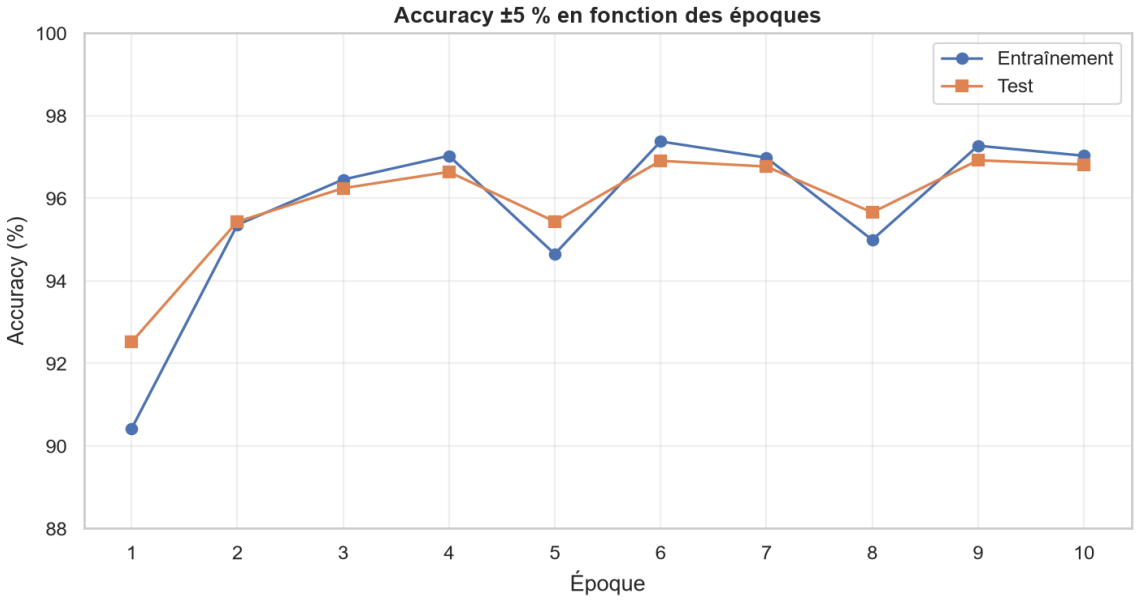


Figure 7 - Pourcentage de prédictions à ±5 % sur l'entraînement et le test.

J'observe que ma loss d'entraînement diminue globalement et que ma loss de test reste supérieure mais proche, sans divergence progressive. Mon accuracy se stabilise autour de 96 à 97 % après quelques époques. J'explique une partie de ses fluctuations par le seuil discret de ±5 % : une petite variation peut faire passer plusieurs exemples juste au-dessus ou juste au-dessous du seuil.

Je ne détecte pas de signe fort d'overfitting sur dix époques. J'observe néanmoins que la progression devient faible après environ six époques, ce qui limite l'intérêt d'un entraînement beaucoup plus long.

8. Conclusion, limites et apprentissages

Conclusion

Mon MLP apprend correctement la relation entre les prix du jour courant et la clôture suivante, mais il ne bat pas ma baseline de persistance. J'en retiens qu'un modèle complexe n'est utile que s'il améliore une référence simple et pertinente.

Limites

- J'interprète le R^2 proche de 1 avec prudence : il mesure surtout la forte similarité entre deux niveaux de prix consécutifs et ne prouve pas que mon modèle prédit correctement les rendements ou la direction du marché.
- J'observe que mes variables OHLC sont très corrélées et que le volume seul apporte peu d'information supplémentaire dans cette formulation.
- J'ai volontairement limité ma GridSearch à un sous-échantillon et à 16 configurations pour rester compatible avec le temps disponible.
- Pour prolonger mon travail, je pourrais prédire un rendement, ajouter une fenêtre temporelle de plusieurs jours ou utiliser un modèle séquentiel.

Ce que ce travail m'a permis d'apprendre

- Inspecter un DataFrame, ses types, ses valeurs manquantes, ses doublons et ses statistiques.
- Distinguer les variables d'entrée de la cible et construire une cible temporelle avec shift.
- Respecter l'ordre chronologique et prévenir les fuites de données.
- Standardiser avec des paramètres appris uniquement sur l'entraînement.
- Comprendre les tensors, mini-batches, couches cachées, fonctions d'activation, loss, gradients et optimiseur.
- Entraîner et évaluer un MLP PyTorch sur GPU local.
- Comparer un réseau à une baseline, lire les courbes train/test et diagnostiquer l'overfitting.
- Interpréter GridSearch, L2, dropout et early stopping sans supposer qu'ils améliorent toujours le modèle.

Mes livrables associés : mon notebook entièrement exécuté, mes tableaux CSV de métriques, mes figures exportées et mon code d'installation reproductible dans le dossier DL-Lab-1.